

Cauã Costa Alves – 855926

Objetivo: Implementar, comparar e justificar as escolhas de projeto de três algoritmos de árvore de decisão:

- ID3 (ganho de informação; atributos categóricos),
- C4.5 (razão de ganho; lida com contínuos),
- CART (índice de Gini; divisões binárias).

Você deverá:

1. implementar os três algoritmos do zero (sem usar DecisionTreeClassifier para treinar; ele pode ser usado apenas como baseline de verificação);
2. explicar todas as decisões técnicas tomadas;
3. mostrar as saídas (árvore aprendida, métricas e artefatos pedidos) para os dataset descritos abaixo.

Observação:

Para todas as listas que envolvem a implementação das funções em Python, solicita-se que:

1. Todas as discussões das questões devem estar contidas na lista. Ou seja, todas as decisões e explicações necessárias precisam estar na lista, que só pode ser entregue em PDF. Listas em qualquer outro formato serão zeradas.
2. Os links para os códigos desenvolvidos devem estar inseridos na lista, com as devidas permissão de acesso. Listas sem permissão de acesso serão zeradas.
3. Este código desta lista deverá estar disponível como biblioteca do python. Mostre como instalar a biblioteca e mostre também todas as saídas dos códigos no próprio PDF.
4. Adicione o link da biblioteca no PDF

Questão 01

Dataset 1 sugerido

Titanic — sobrevivência (Kaggle).

- Target: Survived (0/1)
- Atributos (use ao menos estes): Pclass (1/2/3), Sex, Age, SibSp, Parch, Fare, Embarked (C/Q/S).
- Motivos: mistura de variáveis categóricas e contínuas; classe desbalanceada moderada; fácil interpretação.

Dataset 2 sugerido

Play Tennis (14 linhas, atributos categóricos), clássico para conferir se o ID3/C4.5 estão corretos antes de ir ao Titanic.

Atributos: Outlook (Sunny/Overcast/Rain), Temperature (Hot/Mild/Cool), Humidity (High/Normal), Wind (Weak/Strong).

Classe: Play (Yes/No).

Entregáveis (um único notebook/relatório)

1. Seção 1 — Preparação dos dados

- Titanic: limpeza de missing values (Age, Embarked, etc.).
- Partição: train/ test (ex.: 80/20, estratificada).
- ID3: justificar discretização de contínuos (p.ex., Age, Fare) — discretization simples.
- C4.5/CART: contínuos tratados nativamente por limiares.

2. Seção 2 — Implementações

2.1 Utilidades comuns

- Cálculo de entropia, ganho de informação, razão de ganho, Gini.
- Procura de melhor divisão:
 - Categóricos: por valor (ID3/C4.5) ou binarização ótima (CART).
 - Contínuos: varredura por limiar (ordenar valores únicos; testar pontos médios entre adjacentes).
- Empates (mesmo critério): explicar os critérios de empate

2.2 ID3 (do zero)

- Critério: ganho de informação.
- Atributos categóricos: use o Titanic discretizado.

2.3 C4.5 (do zero)

- Critério: razão de ganho (ganho normalizado pela entropia do split).
- Contínuos: selecionar limiar ótimo; nós multi-ramificados para categóricos.
- Handling de missing: média e moda

2.4 CART (do zero)

- Critério: Gini; splits binários sempre.
- Compare sua árvore com
`sklearn.tree.DecisionTreeClassifier(criterion="gini")`.

3. Seção 3 — O que mostrar como “saídas”

Para cada algoritmo (ID3, C4.5, CART), inclua a árvore gerada e mostre as regras obtidas

Respostas

Link Lista 4.

Seção 1: Código - utilitários (imputação, split, discretização):

```
1  # ===== utils_preprocess.py =====
2  from __future__ import annotations
3  from typing import Tuple
4  import numpy as np
5  import pandas as pd
6
7  def train_test_split_stratified(y: np.ndarray, test_size: float = 0.2, seed: int = 42) -> Tuple[np.ndarray, np.ndarray]:
8      """Split estratificado SÓ pelos rótulos y. Retorna (train_idx, test_idx)."""
9      rng = np.random.default_rng(seed)
10     idx = np.arange(len(y))
11     test_idx = []
12
13     for c in np.unique(y):
14         class_idx = idx[y == c]
15         rng.shuffle(class_idx)
16         n_test = max(1, int(round(test_size * len(class_idx))))
17         test_idx.extend(class_idx[:n_test])
18
19     test_idx = np.array(sorted(test_idx))
20     train_idx = np.array([i for i in idx if i not in set(test_idx)])
21     return train_idx, test_idx
22
23
24 def discretize_equal_frequency(series: pd.Series, bins: int = 4, labels: bool = True) -> pd.Series:
25     """Discretiza por quantis (~mesmo nº de amostras por faixa)."""
26     q = np.linspace(0, 1, bins + 1)
27     edges = np.unique(series.quantile(q).values)
28     edges[0] = -np.inf
29     edges[-1] = np.inf
30     cats = pd.cut(series, bins=edges, include_lowest=True)
31     return cats.astype(str) if labels else cats
32
33
34 def discretize_equal_width(series: pd.Series, bins: int = 4, labels: bool = True) -> pd.Series:
35     """Discretiza por largura fixa (intervalos iguais)."""
36     cats = pd.cut(series, bins=bins, include_lowest=True)
37     return cats.astype(str) if labels else cats
38
39
40 def impute_simple(df: pd.DataFrame) -> pd.DataFrame:
41     """Imputa NaNs: numéricos → mediana; categóricos/objeto → moda."""
42     out = df.copy()
43     for col in out.columns:
44         if out[col].dtype == object:
45             if out[col].isna().any():
46                 out[col] = out[col].fillna(out[col].mode().iloc[0])
47         else:
48             if out[col].isna().any():
49                 out[col] = out[col].fillna(out[col].median())
50     return out
```

Código - Métricas auxiliares:

```
# ===== metrics_aux.py =====
from __future__ import annotations
import numpy as np

def accuracy(y_true, y_pred) -> float:
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    return float((y_true == y_pred).mean())

def confusion_matrix(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    labels = sorted(list(set(y_true) | set(y_pred)))
    L = len(labels)
    lab2i = {lab: i for i, lab in enumerate(labels)}
    m = np.zeros((L, L), dtype=int)
    for t, p in zip(y_true, y_pred):
        m[lab2i[t], lab2i[p]] += 1
    return labels, m
```

Seção 2:

2.1) Métricas

- Entropia
- Ganho de informação
- Razão de ganho (C4.5)
- Gini (CART)

Como achar o melhor split:

- Categóricos
 - ID3/C4.5: um ramo por valor.
 - CART: sempre binário → busca a melhor bipartição das categorias.
- Contínuos
 - Ordena valores únicos, testa limiares nos pontos médios adjacentes.
 - Escolhe o que maximiza Gain/GainRatio (ID3/C4.5) ou minimiza Gini (CART).

- Empates (mesma pontuação)
 1. prefere split mais simples (menos ramos/profundidade),
 2. depois maior nº de amostras válidas,
 3. em C4.5, menor SplitInfo,
 4. se ainda empatar, ordem alfabética do atributo (desempate determinístico).

2.2) ID3 (ganho de informação; atributos categóricos)

<https://github.com/cauacostaalves/PUCMG/tree/e165801ee03bc001cc77c05065bfc1f224d6b42a/CienciasDaComputa%C3%A7%C3%A3o/Quarto%20Semestre/IA/Listas/Lista%204/Codigos/ID3>

2.3) C4.5 (razão de ganho; contínuos por limiar; categórico multi-ramo)

<https://github.com/cauacostaalves/PUCMG/tree/e165801ee03bc001cc77c05065bfc1f224d6b42a/CienciasDaComputa%C3%A7%C3%A3o/Quarto%20Semestre/IA/Listas/Lista%204/Codigos/C4.5>

2.4) CART (índice de Gini; divisões binárias; categórico por subconjunto)

<https://github.com/cauacostaalves/PUCMG/tree/e165801ee03bc001cc77c05065bfc1f224d6b42a/CienciasDaComputa%C3%A7%C3%A3o/Quarto%20Semestre/IA/Listas/Lista%204/Codigos/CART>

Seção 3:

3.1) ID3

Acurácia (treino): 0.8808

Acurácia (teste): 0.8034

Matriz de confusão (teste):

labels: 0, 1

| | 0 | 1 |

| - | - | - |

| 0 |99|11|

| 1 |24|44|

Árvore:

```
[Sex]
├─ female
│   └─ [Pclass]
│       ├── 1
│       │   └─ [Fare]
│       │       ├── (14.454, 30.5] → Folha: 1
│       │       └─ (30.5, inf] → Folha: 1
│       ├── 2
│       │   └─ [Age]
│       │       ├── (inf, 22.0] → Folha: 1
│       │       └─ (22.0, 28.0] → Folha: 1
│       └─ 3
│           └─ [Fare]
│               ├── (7.896, 14.454] → Folha: 1
│               └─ (14.454, 30.5] → Folha: 1
└─ male
    └─ [SibSp]
        ├── 0
        │   └─ [Pclass]
        │       ├── 1 → Folha: 0
        │       ├── 2 → Folha: 0
        │       └─ 3 → Folha: 0
        └─ ≥1 → Folha: 0
```

3.2) C4.5

Acurácia (treino): 0.8219

Acurácia (teste): 0.8202

Matriz de confusão (teste):

labels: 0, 1

| | 0 | 1 |

|-| - | - |

|0|103| 7|

|1| 25 |43|

Árvore:

```
[Sex]
├ female
│   [SibSp < 6]
│   ├── True
│   │   [Pclass < 2.5]
│   │   ├── True
│   │   │   [Fare < 28.8562]
│   │   │   ├── True
│   │   │   │   [Age < 53.5]
│   │   │   │   ├── True → Folha: 1
│   │   │   │   └ False → Folha: 1
│   │   │   └ False → Folha: 1
│   │   └ False → Folha: 1
│   └ False → Folha: 1
└ male
    [Age < 1.5]
    ├── True → Folha: 1
    └ False
        [Fare < 387.665]
        ├── True
        │   [SibSp < 4.5]
        │   ├── True
        │   │   [Parch < 2.5]
        │   │   ├── True
        │   │   │   [Pclass < 1.5] → Folha: 0
        │   │   │   └ False → Folha: 0
        │   │   └ False → Folha: 0
        │   └ False → Folha: 0
        └ False → Folha: 0
```

3.3) CART

Acurácia (treino): 0.8682

Acurácia (teste): 0.8258

Matriz de confusão (teste):

labels: 0, 1

| | 0 | 1 |

|-| - | - |

|0|100|10|

|1| 21 |47|

Árvore:

```
[Sex ∈ {'female'}]
├ True
│   [Pclass < 2.5]
│   ├── True
│   │   [Fare < 28.8562]
│   │   ├── True
│   │   │   [Age < 53.5]
│   │   │   ├── True → Folha: 1
│   │   │   └ False → Folha: 1
│   │   └ False → Folha: 1
│   └ False
│       [Fare < 20.6625]
│       ├── True
│       │   [Age < 7]
│       │   ├── True → Folha: 1
│       │   └ False
│       │       [Fare < 8.0396]
│       │       ├── True
│       │       │   [Age < 29.25]
│       │       │   ├── True → Folha: 1
│       │       │   └ False → Folha: 0
│       │       └ False
│       │           [Fare < 15.8]
│       │           ├── True → Folha: 0
│       │           └ False → Folha: 0
│       └ False → Folha: 1
└ False
    [Parch < 0.5]
    ├── True
    │   [Age < 5.5]
    │   ├── True
    │   │   [Age < 3.5] → Folha: 0/1 (dependendo do conjunto)
    │   └ False → Folha: 0
    └ False → Folha: 0
```


